

DA5400 (July-Nov 2025)
Assignment 1 Report
Nishchala Mukku (EE24S004)

Q1:

Refer to **Q1.m** file. You are given a dataset with 1000 data points each in $\mathbb{R}^2$.

- (a) **Write a piece of code to run the PCA algorithm on this data-set. How much of the variance in the data-set is explained by each of the principal components?**

Sol:

- Principal Component Analysis (PCA) is a dimensionality reduction technique that identifies new orthogonal axes (principal components) along which the data variance is maximized.
- We take the features in the dataset which exists in multiple dimensions and projecting it on a new dimension (vector) where each feature takes up as much space as possible in order to preserve the variance of the original dataset.
- The first principal component (PC1) corresponds to the direction of maximum variance; the second principal component (PC2) is orthogonal to PC1 and explains the next highest variance and so on.
- Before applying PCA, the dataset must be centered (mean of each feature subtracted).

Algorithm

- (a) We need to center the dataset, $X_i \in \mathbb{R}^{m \times d}$. This is done by subtracting the mean. Compute the mean of each feature:

$$\mu_m = \frac{1}{m} \sum_{i=1}^m x_i, \quad \mu \in \mathbb{R}^d$$

Make a row vector μ consisting of means of each features as the data entries. then subtract the mean from the data:

$$X_c = X - \mu,$$

- (b) The covariance matrix of the centered data is

$$C = \frac{1}{m} X_c^\top X_c.$$

- (c) Then we do eigen decomposition to get the eigen vectors and the corresponding eigen values of the covariance matrix of the centered data. This can be mathematically written as,

$$Cv_i = \lambda_i v_i,$$

where λ_i are eigenvalues and v_i are eigenvectors. Sort the eigenvalues in descending order:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d.$$

The corresponding eigenvectors v_i are the principal component directions in decreasing order of the importance.

- (d) The proportion of variance, which is known as Explained Variance Ratio, explained by the i -th principal component is:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}.$$

Higher EVR values indicate more significant components. As a rule of thumb we take in 95% of the principal components of the whole lot of principal components we get after analysis and discard the remaining components.

- (e) To transform the dataset into the PC space:

$$Z = X_c V,$$

where $V = [v_1, v_2, \dots, v_k]$ contains the top- k eigenvectors. Each row of Z represents a sample in the new PC coordinate system.

- (f) To see how well our PCA is, we can reconstruct the data using these components. Using only the top- k components, the data can be approximately reconstructed as:

$$X_{\text{recon}} = ZV^\top.$$

The reconstruction error decreases as more PCs are included, and becomes zero if all d PCs are used.

Methodology

- (a) The dataset (`dataset1-assignment1-Sheet1.csv`) containing 1000 points in $\mathbb{R}^2$ was loaded.
- (b) The data was centered by subtracting the mean of each column.
- (c) The covariance matrix was computed manually from the centered data.
- (d) Eigenvalues and eigenvectors of the covariance matrix were obtained using eigen-decomposition.

- (e) Eigenvalues were sorted in descending order, and eigenvectors were reordered accordingly.
- (f) The explained variance ratio (EVR) was computed as $\lambda_i / \sum \lambda_j$.
- (g) The centered data was projected onto the principal components to obtain new coordinates in the PC basis.
- (h) Reconstruction of the original data was performed from PCs to evaluate reconstruction accuracy and compute the mean squared error (MSE).

Plots

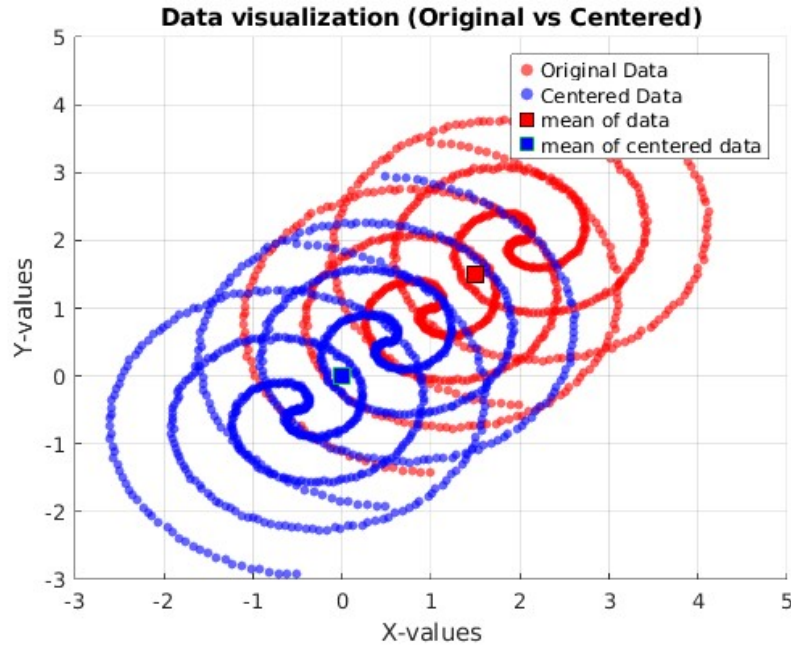


Figure 1: Centered data scatter plot. The mean of the dataset is shifted to the origin.

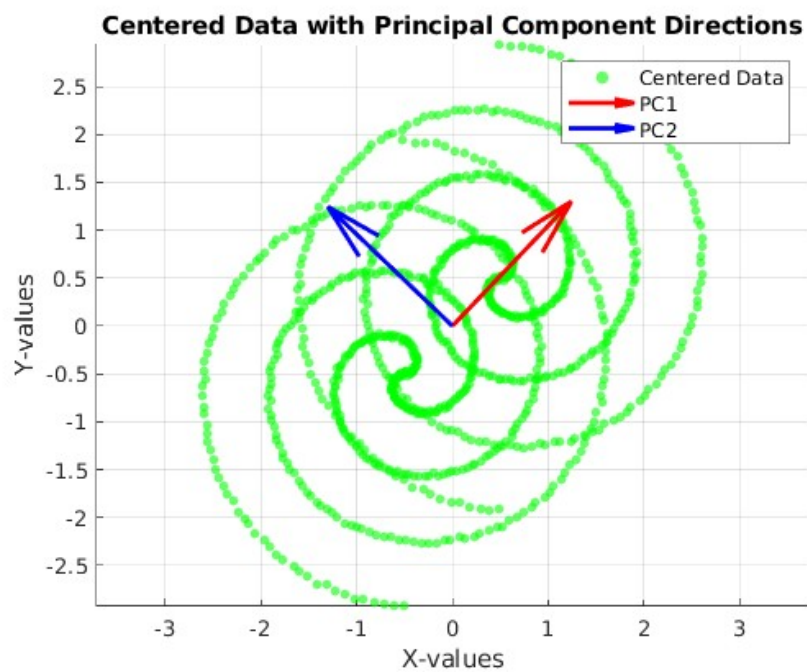


Figure 2: Centered data with principal component directions. PC1 (red) captures maximum variance, PC2 (blue) is orthogonal and captures the remaining variance.

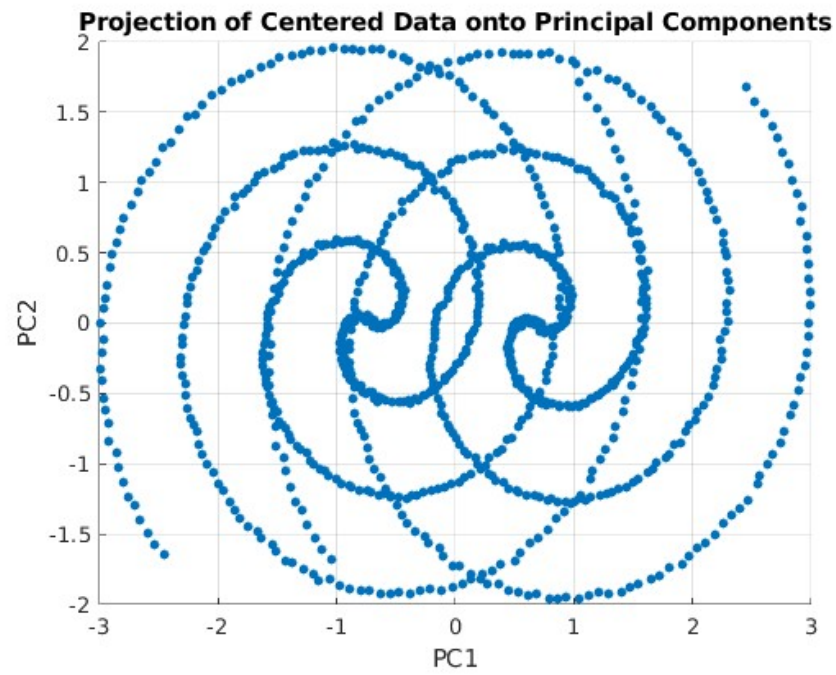


Figure 3: Projection of centered data onto PC1 and PC2. The new axes represent maximum variance directions.

```
>> Q1
Ratio by each principal component:
0.6539
0.3461
```

Figure 4: The first principal component (PC1) accounts for approximately 65.39% of the variance, while the second principal component (PC2) accounts for 34.61%.

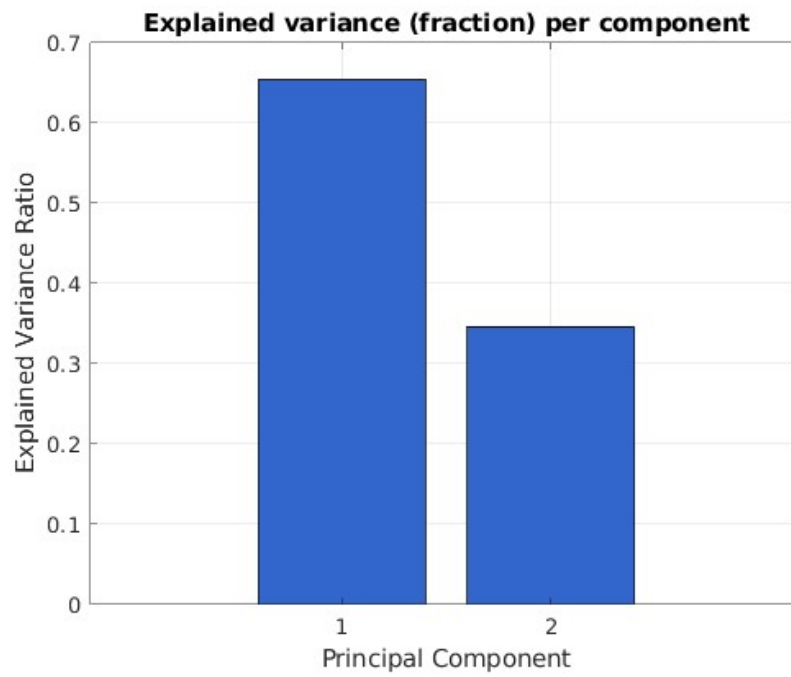


Figure 5: Explained variance ratio (EVR) bar plot. PC1 explains a larger fraction of the variance compared to PC2.

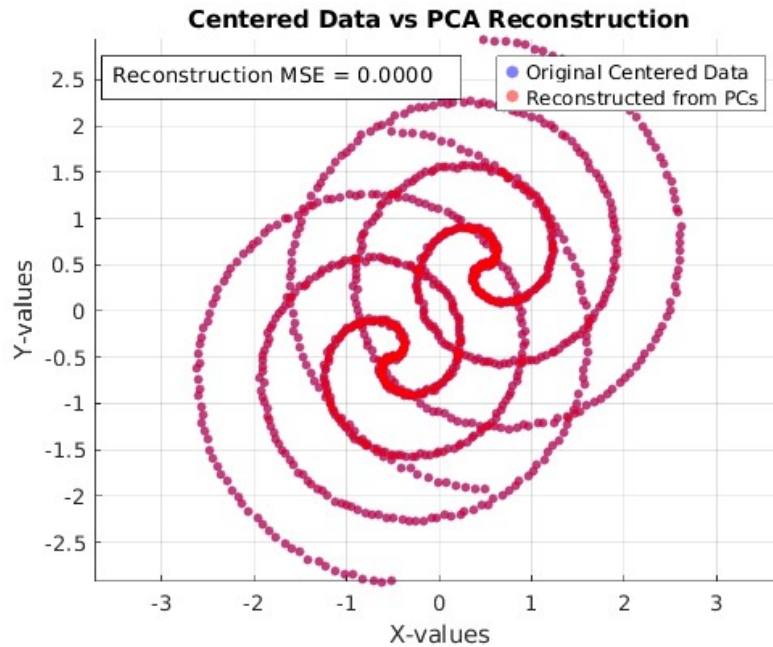


Figure 6: Comparison of original centered data (blue) and PCA reconstruction (red). Using both PCs, reconstruction error (MSE) is nearly zero. This implies, we can't make clusters effectively. Here the non-linearity of the data isn't handled well by linear PCA.

Conclusion: PCA applied to the dataset shows that the first principal component explains **65.39%** of the variance, while the second explains **34.61%**. Thus, both components contribute significantly, and retaining both is necessary to preserve the structure of the data. Also, it can be seen that PCA isn't able to separate out the data and no proper clustering can be done.

- (b) **Write a piece of code to implement the Kernel PCA algorithm on this dataset. Explore various kernels discussed in class. For each Kernel, plot the projection of each point in the dataset onto the top-2 principal components. Use one plot for each kernel - In case of RBF kernel, use a different plot for each value of σ that you use.**

Sol:

- Principal Component Analysis (PCA) is limited to finding *linear* projections. If the dataset lies on a non-linear manifold (e.g., spirals, circles), linear PCA may fail to separate the structure.
- Kernel Principal Component Analysis (Kernel PCA) is an extension

of PCA that enables non-linear dimensionality reduction by implicitly mapping the data into a higher-dimensional feature space using kernel functions.

- Kernel PCA extends PCA by mapping the data into a high-dimensional feature space $\phi(x)$ and then applying PCA in this space.
- Instead of explicitly computing $\phi(x)$, the *kernel trick* is used: the dot product in feature space is replaced by a kernel function $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$.
- Common kernels include:
 - Polynomial kernel: $K(x_i, x_j) = (x_i^\top x_j + 1)^d$
 - Gaussian RBF kernel: $K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

Algorithm

- (a) Given a dataset $X = \{x_1, x_2, \dots, x_m\}$ with $x_i \in \mathbb{R}^d$, choose a kernel function

$$K(x_i, x_j) : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}.$$

- (b) Construct the kernel matrix $K \in \mathbb{R}^{m \times m}$ with entries

$$K_{ij} = K(x_i, x_j).$$

- (c) Even if we center the original dataset, after mapping through ϕ , the transformed data $\phi(x)$ may not be centered. We therefore compute the centered kernel K_c as:

$$K_c(i, j) = K(i, j) - \frac{1}{m} \sum_{\ell=1}^m K(i, \ell) - \frac{1}{m} \sum_{\ell=1}^m K(j, \ell) + \frac{1}{m^2} \sum_{\ell=1}^m \sum_{\ell'=1}^m K(\ell, \ell').$$

- (d) Solve the eigenvalue problem on the centered kernel:

$$K_c \beta = \lambda \beta,$$

where λ are eigenvalues and β are eigenvectors.

- (e) Normalize eigenvectors to obtain principal directions in feature space:

$$\alpha_k = \frac{\beta_k}{\sqrt{n \lambda_k}}.$$

- (f) Project a data point x_i onto the k -th principal component in feature space:

$$z_{ik} = \sum_{j=1}^m \alpha_{jk} K(x_i, x_j).$$

- (g) Collect the top- k components to form the embedding:

$$Z = K_c \cdot [\alpha_1, \alpha_2, \dots, \alpha_k].$$

Methodology

- (a) The dataset (`dataset1-assignment1-Sheet1.csv`) was first centered.
- (b) Three types of kernels were applied:
 - Polynomial kernels: degrees $d = 2$ to 5.
 - RBF kernels: σ varied from 0.5 to 5.0 in steps of 0.5.
- (c) Eigen-decomposition of the kernel matrices was performed, and the top two principal components were used for visualization.
- (d) Scatter plots of data projected onto the new feature space were generated to visualize separability.

Plots

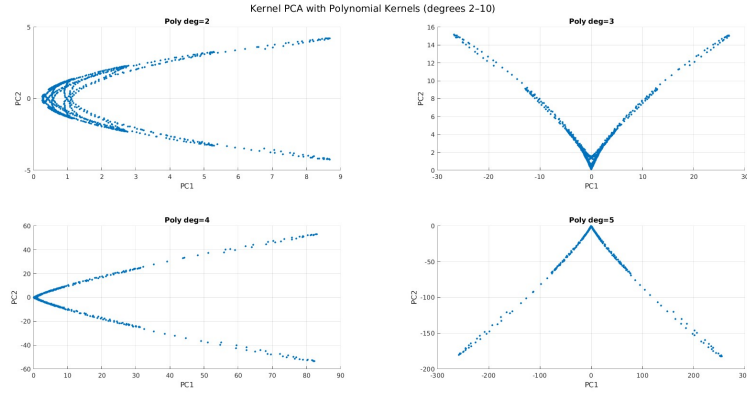


Figure 7: Kernel PCA with polynomial kernels (degrees 2–10). For moderate degrees (3–4), separation improves, but very high degrees distort the data and cause instability.

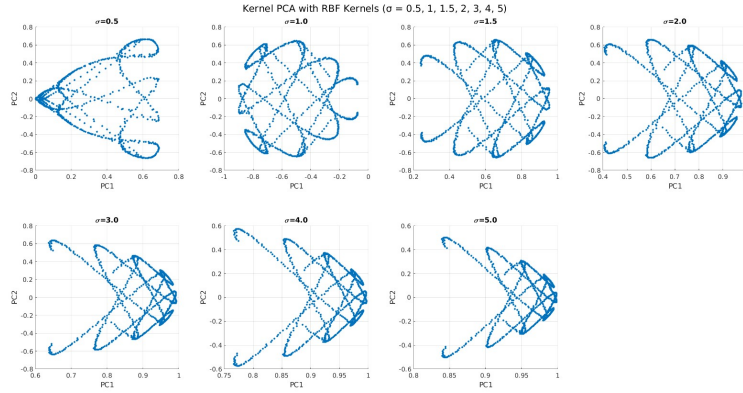


Figure 8: Kernel PCA with RBF kernels ($\sigma = 0.5$ to 5.0). For $\sigma \approx 2.0$, the spirals are well-separated in the feature space. Too small σ overfits, too large σ oversmooths.

(c) **Which Kernel do you think is best suited for this dataset and why? Sol:**

- The dataset consists of two intertwined spirals, which are highly non-linear in structure.
- PCA: Equivalent to standard PCA. It fails to separate the spirals, since the data is not linearly separable. This can be seen in fig 9. The linear kernel performs identically to PCA and fails to separate non-linear spiral data. Also, the first PC, doesn't reproduce or cover the non linearity of the data and only 20% data fits. This can be seen in fig 10.

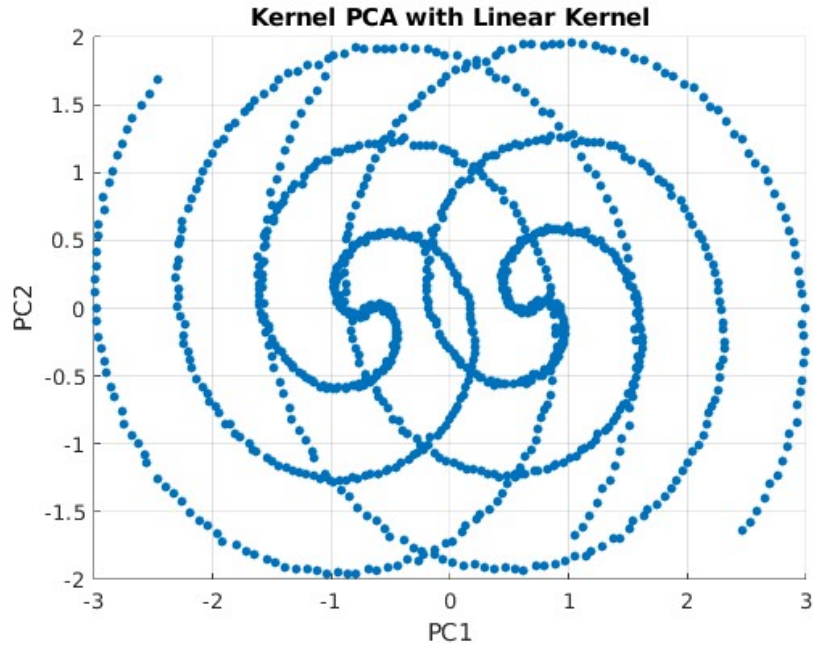


Figure 9: Kernel PCA with linear kernel. The two spirals remain entangled, showing that linear PCA cannot separate non-linear structures.

- The linear kernel failed to separate the clusters, since the dataset is non-linearly separable. The projections in the linear kernel space retained the spiral overlap, offering no significant improvement over standard PCA.
- The RBF kernel exaggerated the spiral geometry, producing complex, winding structures in the transformed space. Although faithful to the data's local neighborhoods, this mapping made the clusters more complicated and harder for K-means to separate effectively.
- Conclusion: The polynomial kernel with low degree ($d = 2$ or 3) is the best suited for this dataset, as it balances nonlinearity with smoothness, yielding well-separated clusters. In contrast, the RBF kernel overemphasized the spiral structure and the linear kernel failed to capture any nonlinear separation.

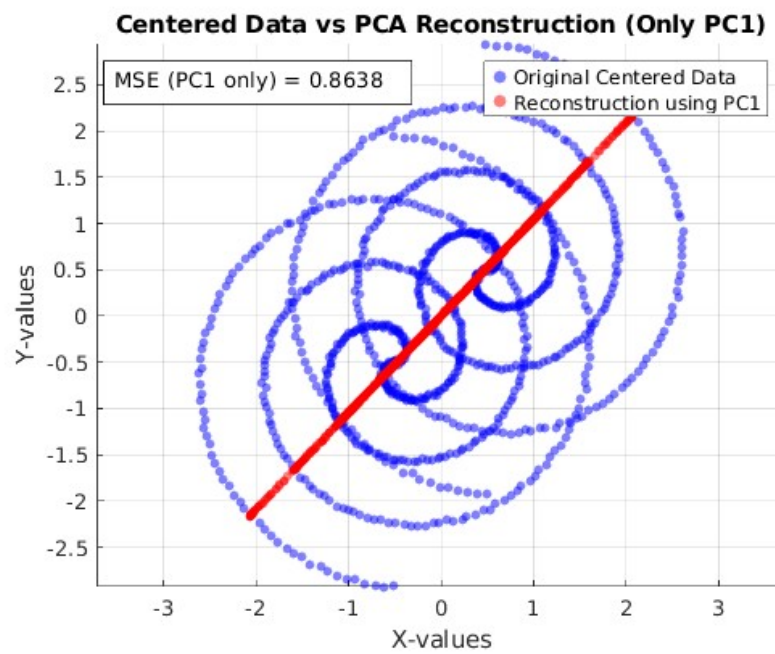


Figure 10: 1st PC is able to recover only 20% data and the non-linearity is not picked up

Q2:

Refer to **Q2.m** file. You are given a dataset with 1000 data points each in $\mathbb{R}^2$.

- (a) Write a piece of code to run the algorithm studied in class for the K-means problem with $k = 4$. Try 5 different random initialization and plot the error function w.r.t. iterations in each case. In each case, plot the clusters obtained in different colors.

Sol:

- K-means clustering is an unsupervised learning algorithm that partitions n data points into k clusters, where each point belongs to the cluster with the nearest mean (called the cluster center or centroid).
- This iterative procedure continues until convergence, which can be defined by:
 - Assignments no longer change, or
 - The decrease in J between iterations falls below a threshold.

Algorithm

- (a) Choose the number of clusters K , a maximum number of iterations `max_iter`, and a tolerance $\text{tol} > 0$ for centre movement.
- (b) Initialization: select K distinct initial centers by random sampling from the dataset:

$$\{\mu_1^{(0)}, \dots, \mu_K^{(0)}\} = \{x_{r_1}, \dots, x_{r_K}\}, \quad r_j \text{ from a random permutation of } \{1, \dots, m\}.$$

- (c) Repeat for $t = 1, 2, \dots, \text{max_iter}$:

- i. Assignment step: For each data point x_i assign it to the nearest centre

$$z_i^{(t)} = \arg \min_{k \in \{1, \dots, K\}} \|x_i - \mu_k^{(t-1)}\|_2^2.$$

Denote the cluster sets $C_k^{(t)} = \{i : z_i^{(t)} = k\}$.

- ii. Update step: for each cluster k recompute the centre as the mean of its assigned points

$$\mu_k^{(t)} = \begin{cases} \frac{1}{|C_k^{(t)}|} \sum_{i \in C_k^{(t)}} x_i, & \text{if } |C_k^{(t)}| > 0, \\ \text{reinitialize } \mu_k^{(t)} \text{ to a randomly chosen data point,} & \text{if } |C_k^{(t)}| = 0. \end{cases}$$

- (d) Stopping criterion: stop when either

- the assignments do not change: $z^{(t)} = z^{(t-1)}$, or

- the maximum centre movement is below tolerance:

$$\max_k \|\mu_k^{(t)} - \mu_k^{(t-1)}\|_2 < \text{tol},$$

or the iteration count reaches `max_iter`. This is also called as Objective function.

Methodology

- The dataset (`Dataset2-Assignment1.csv`) with 1000 points in $\mathbb{R}^2$ was loaded and centered.
- K-means clustering was applied with $K = 4$ clusters.
- The algorithm was run with 5 different random initializations.
- For each run:
 - The cluster assignments and centroids were obtained.
 - The error function J was plotted against iterations.
 - The clustered data was visualized with each cluster in a different color.

Plots

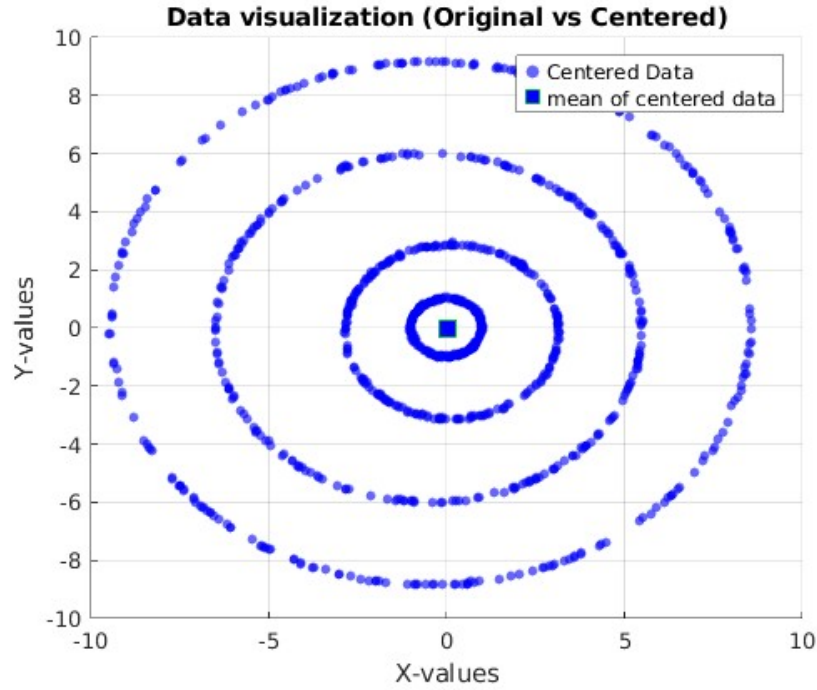


Figure 11: Centered data scatter plot. The mean of the dataset is shifted to the origin.

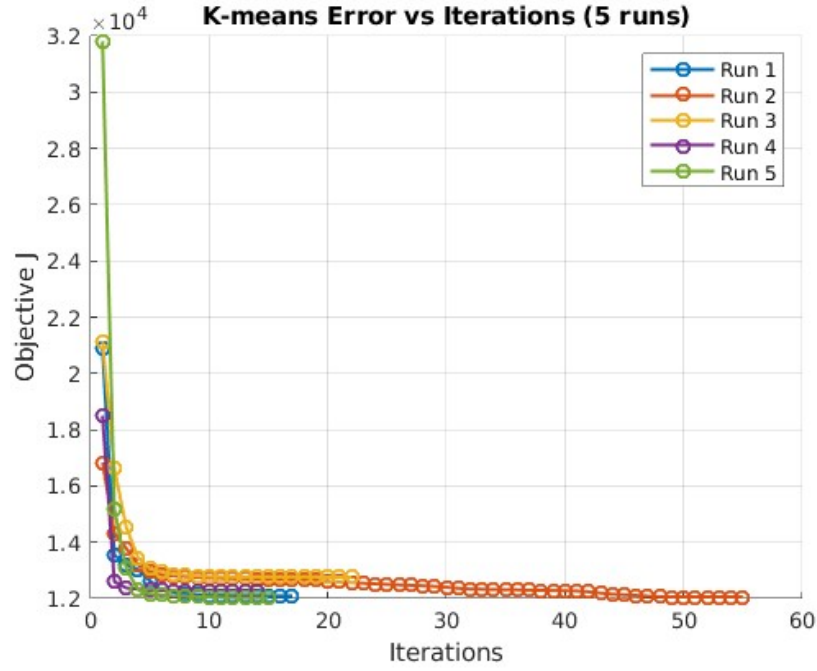


Figure 12: Error function J versus iterations for different initializations. Each curve shows convergence behavior for one initialization.

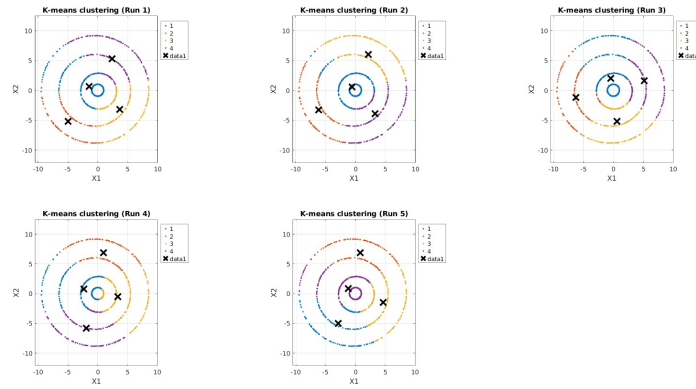


Figure 13: K-means clustering result with $K = 4$. Different colors represent different clusters; black crosses denote final centroids.

Conclusion: The K-means algorithm with $K = 4$ converges reliably within

a few iterations, though the objective function varies slightly across different random initializations. The clusters obtained are compact and well-separated, validating the suitability of K-means for this dataset.

- (b) **Fix a random initialization. For $K = 2, 3, 4, 5$, obtain cluster centers according to K-means algorithm using the fixed initialization. For each value of K , plot the Voronoi regions associated to each cluster center. (You can assume the minimum and maximum value in the data-set to be the range for each component of $\mathbb{R}^2$).**

Sol:

- Once cluster centers are obtained from the K-means algorithm, the partition of the input space can be visualized by Voronoi regions.
- A Voronoi region corresponding to a center μ_k is defined as:

$$V_k = \{x \in \mathbb{R}^2 : \|x - \mu_k\|_2 \leq \|x - \mu_j\|_2, \forall j \neq k\}.$$

- Each region V_k consists of all points in the plane that are closer to μ_k than to any other cluster center. These regions form a complete partition of $\mathbb{R}^2$ into K disjoint convex polygons.
- The Voronoi diagram thus provides a geometric interpretation of the decision boundaries formed by the K-means algorithm.
- By varying K , we can visualize how the complexity of the partition changes and how well the model fits the dataset.

Algorithm

- (a) Fix a random initialization of cluster centers $\{\mu_1, \mu_2, \dots, \mu_K\}$.
- (b) For each $K \in \{2, 3, 4, 5\}$:
 - i. Run the K-means algorithm until convergence to obtain the final cluster centers.
 - ii. For the obtained cluster centers, compute the Voronoi diagram using:

$$V_k = \{x \in \mathbb{R}^2 : \|x - \mu_k\|^2 \leq \|x - \mu_j\|^2, \forall j \neq k\}.$$

- iii. Plot the original dataset points, colored according to cluster assignments.
 - iv. Overlay the Voronoi boundaries to show the decision regions.
- (c) Compare how the Voronoi regions evolve as K increases from 2 to 5.

Methodology

- (a) A fixed random initialization was chosen to ensure consistency across different K values.

- (b) The K-means algorithm was executed separately for $K = 2, 3, 4, 5$ on the centered dataset.
- (c) For each K , the final cluster centers were obtained, and the Voronoi diagram was computed using MATLAB's `voronoi` function.
- (d) The dataset points were colored based on their cluster assignment, and Voronoi boundaries were overlaid to visualize the partition.

Plots

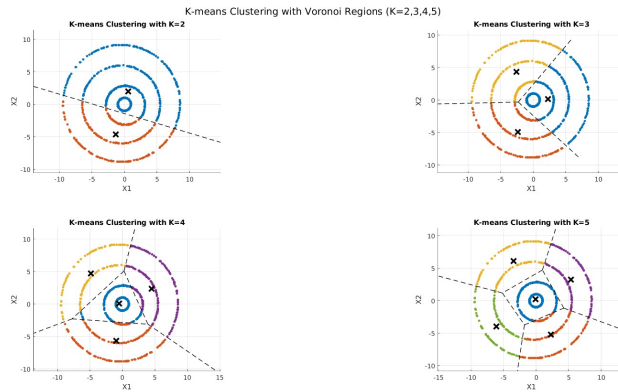


Figure 14: Voronoi regions for $K = \{2, 3, 4, 5\}$ with fixed initialization. Data points are colored by cluster assignment.

- For $K = 2$, the space is divided into two large regions, which oversimplifies the dataset.
- For $K = 3$ and $K = 4$, the partitions align more closely with the natural structure of the dataset.
- For $K = 5$, the regions become more refined but risk over-partitioning if clusters overlap.

Conclusion: The Voronoi diagrams provide a geometric visualization of the K-means partitions. As K increases, the boundaries become finer and better capture the data distribution, but an overly large K may lead to overfitting and poor generalization.

- (c) **Run the spectral clustering algorithm (spectral relaxation of K-means using Kernel- PCA) $k = 4$. Choose an appropriate kernel for this data-set and plot the clusters obtained in different colors. Explain your choice of kernel based on the output you obtain.**
Sol: Theory:

- Spectral clustering leverages the eigen-decomposition of a similarity (kernel) matrix to map data into a low-dimensional space where clusters become more separable.
- Instead of clustering directly in the input space, we compute a kernel matrix K that measures similarity between pairs of points.
- Eigen-decomposition of K provides principal directions in the feature space. Selecting the top- k eigenvectors yields an embedding where conventional K-means can be applied to obtain clusters.
- The choice of kernel plays a crucial role in capturing the geometry of the data:
 - RBF kernel:

$$K_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right),$$

which captures local similarity. The parameter σ controls the width of the Gaussian and hence the sensitivity to local structure.

- Polynomial kernel:

$$K_{ij} = (x_i^\top x_j + c)^d,$$

which captures global nonlinear structures. The degree d determines the complexity of the decision boundary.

Algorithm:

- (a) Construct the kernel matrix K using either RBF or polynomial kernel.
- (b) Perform eigen-decomposition of K and sort eigenvalues in descending order.
- (c) Select the top- k eigenvectors to form the embedding matrix $U \in \mathbb{R}^{n \times k}$.
- (d) Normalize rows of U so that each data point is represented as a unit vector.
- (e) Run K-means clustering on the rows of U to obtain k clusters.
- (f) Visualize the resulting clusters in the original feature space.

Methodology:

- For the RBF kernel, several σ values were tested. The best separation was observed at $\sigma = 2$, which produced compact and well-separated clusters.
- For the polynomial kernel, degrees $d = 3$ was tested. Moderate values such as $d = 3$ provided smoother separation, while higher degrees led to overly complex boundaries and less clear separation.

Plots:

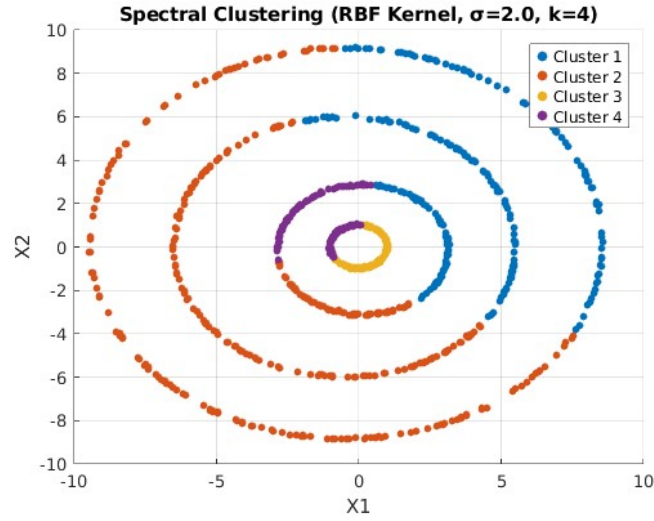


Figure 15: Spectral clustering with RBF kernel ($\sigma = 2$, $k = 4$). Clusters are well-separated, demonstrating the effectiveness of the RBF kernel for capturing local structures.

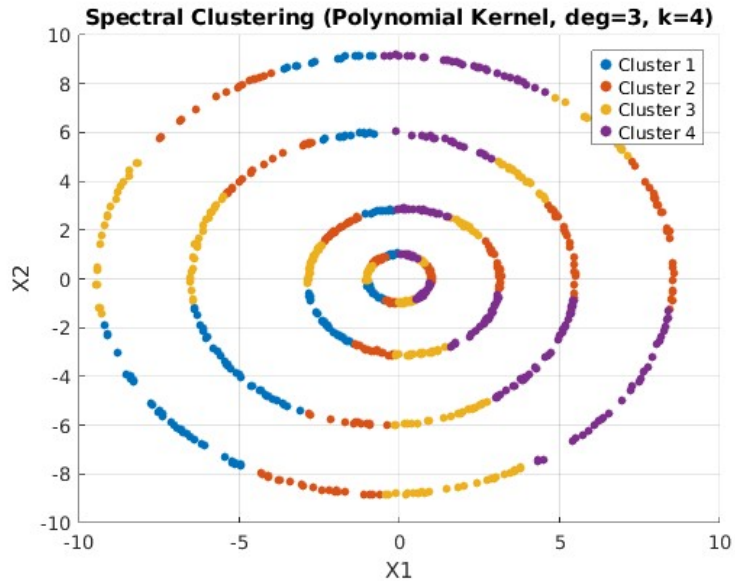


Figure 16: Spectral clustering with Polynomial kernel ($d = 3$, $k = 4$).

Conclusion:

- The RBF kernel provided the best clustering results for this dataset, with $\sigma = 2$ yielding compact and well-separated clusters.
 - The polynomial kernel could also separate clusters, but its performance was sensitive to the degree parameter and often less stable.
 - Hence, for this dataset, the RBF kernel is the most suitable choice due to its ability to model local nonlinear structures effectively.
- (d) **Instead of using the method suggested by spectral clustering to map eigenvectors to cluster assignments, use the following method: Assign data point i to cluster ℓ whenever**

$$\ell = \arg \max_{j=1,\dots,k} v_{ji},$$

where $v_j \in \mathbb{R}^n$ is the eigenvector of the Kernel matrix associated with the j -th largest eigenvalue. How does this mapping perform for this dataset? Explain your insights.

- In standard spectral clustering, after constructing the kernel matrix and obtaining its eigen-decomposition, the top- k eigenvectors form an embedding matrix U .
- Typically, K-means is run on the rows of U to obtain cluster assignments.
- In this variant, we instead assign each data point i to the cluster ℓ according to:

$$\ell = \arg \max_{j=1,\dots,k} v_j(i),$$

where v_j is the j -th eigenvector of the kernel matrix, and $v_j(i)$ is its i -th entry.

- This method avoids running K-means, relying purely on the spectral information from eigenvectors.

Algorithm:

- (a) Construct the kernel matrix K (RBF kernel with $\sigma = 2$ was used).
- (b) Perform eigen-decomposition of K and sort eigenvalues and eigenvectors in descending order.
- (c) Select the top- k eigenvectors $\{v_1, v_2, \dots, v_k\}$.
- (d) For each data point i , assign the label:

$$\text{label}(i) = \arg \max_j v_j(i).$$

- (e) Visualize the clusters in the original 2D space.

Observations:

- This approach indeed partitions the data into clusters without the need for K-means.
- However, one of the clusters may be skipped because no data points have the maximum contribution in that eigenvector. In our experiment, cluster **1** was skipped, and only clusters **2, 3, and 4** were populated.
- This highlights a limitation of the method: the eigenvectors do not always distribute their dominance evenly across data points, which can cause imbalance or missing clusters.

Plots:

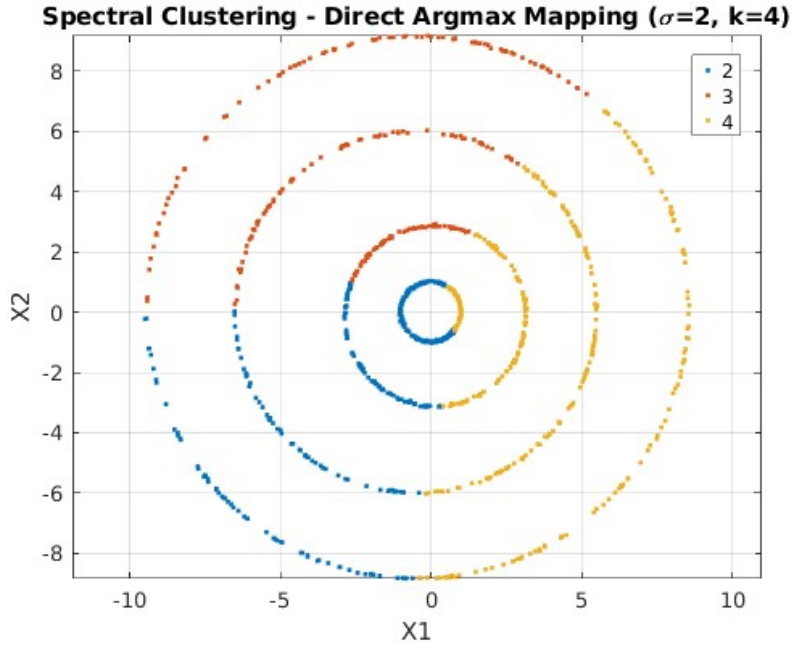


Figure 17: Cluster assignment using the maximum eigenvector component rule. Only clusters 2–4 are assigned, while cluster 1 is skipped.

Conclusion:

- This eigenvector-based mapping provides a direct way of obtaining clusters but lacks robustness.
- Compared to the K-means-based spectral clustering, this approach may skip clusters and yield unbalanced partitions.

- For this dataset, the standard spectral clustering (with K-means) is preferable, as it consistently provides well-separated and balanced clusters.